

Liquid Oxygen Inventory Routing

Ilyas Mammadov Dogukan Murat Kesse

1 Introduction

Air Liquide is a global leader in the production and distribution of industrial and medical gases, including liquid oxygen (LOX) and liquid nitrogen (LIN). These cryogenic products are stored at extremely low temperatures and transported using specialized vacuum-insulated tanker trailers. The distribution of such products constitutes a critical supply chain operation, as uninterrupted availability is essential — particularly for medical facilities where oxygen supply directly impacts patient safety.

A distinguishing characteristic of this supply chain is its operation under a Vendor Managed Inventory (VMI) policy. Unlike traditional supply chains where customers place replenishment orders, Air Liquide retains full responsibility for monitoring customer inventory levels via remote observing systems and proactively scheduling deliveries. Customers do not initiate orders; instead, the supplier must determine when each customer requires a visit, how much product to deliver, and which vehicle to dispatch. This approach creates complex integrated decision-making problems that include inventory management, vehicle routing, and workforce scheduling.

The problem is the design of an optimal distribution plan over a **multi-day planning horizon of 30 days** for a cryogenic liquid oxygen supply chain. The distribution network consists of three categories of nodes:

- **Bases:** Depot locations where the vehicle fleet and driver workforce are stationed. All driver shifts must originate from and terminate at a base.
- **Sources:** Production plants where tanker trailers are loaded with liquid oxygen prior to delivery.
- **Customers:** Demand sites equipped with on-site cryogenic storage tanks. Each customer consumes oxygen at a known, time-varying rate. The amount of oxygen they consume is provided as hourly demand forecasts, and their tank inventory must always remain between zero and maximum storage capacity.

The physical and regulatory characteristics of the problem requires the following constraints on the distribution plan:

1. **Vehicle–Customer Compatibility:** Due to physical dimensions of equipment and on-site safety regulations, only a designated subset of trailers is permitted to access each customer location. This trailer-to-customer compatibility mapping must be strictly met.
2. **Heterogeneous Fleet:** Trailers differ in their carrying capacities and per-kilometer operating costs. Each driver is assigned to a specific trailer, forming a driver–trailer pair that operates as a unit.
3. **Driver Work Regulations:** Drivers are subject to maximum driving duration limits per shift and mandatory minimum rest periods between consecutive shifts. Each driver has defined availability windows in terms of minutes specifying the time intervals during which they may operate.
4. **Inventory Feasibility:** Customer tank inventory levels must remain non-negative (no stockouts) and must not exceed the tank's maximum storage capacity at any point during the planning horizon.
5. **Shift Structure:** Every operational shift must begin at a base, proceed to a source for product loading, make deliveries to one or more customers, and return to a base before the shift's time limit is reached.

The performance of the distribution plan is evaluated by the **Logistics Ratio (LR)**,

$$\min Z = \frac{\sum(\text{Distance} \times \text{Distance Cost}) + \sum(\text{Time} \times \text{Time Cost})}{\sum \text{Quantity Delivered}}$$

The numerator captures the total operational cost, comprising distance-dependent vehicle operating costs and time-dependent driver wage costs. The denominator represents the total volume of product successfully delivered to customers over the planning horizon.

This fractional objective function enforces a balance between route efficiency and delivery utility. A distribution plan that minimizes cost by making few deliveries will be penalized by a small denominator, while a plan that delivers large quantities through inefficient routing will be penalized by a large numerator. The optimal plan must therefore achieve the lowest possible unit cost of delivery.

2 Literature Review

Govindan (2013) stated that vendor managed inventory (VMI) allows the vendor to control replenishment decisions while the buyer shares demand information. In a standard supply chain structure, the member stages of the supply chain make decisions locally without considering the effect of these decisions on the other stages of the supply chain. This process can also be called local cost optimization of each stage. A supply chain comprises different members which have one common goal, delivery of the finished product to the end customer. Generally, the supply chain starts with the raw materials vendor and ends with the downstream stage which provides the product to the end customer. Vendor managed inventory (VMI) is a practice through which the vendor takes over the replenishment policy of the buyer and decides upon replenishment quantities and timings as to maintain the inventory within specified minimum and maximum limits. In return, the buyer shares the sales data or forecasts and keeps the vendor informed with the upcoming events.

2.1 Inventory Routing Problem

The Inventory Routing Problem (IRP) originally appeared in real industrial applications during the early 1980s, particularly in the distribution operations of Air Products and Chemicals (André et al., 2020). Since then, it has become one of the most widely studied integrated logistics problems because it combines inventory management and vehicle routing within a single decision framework. A major milestone in the development of the field was the study by Campbell, Clarke, Kleywegt, and Savelsbergh (1998), which introduced the classical single-product, multi-period IRP formulation that is still widely referenced today. Their paper also showed that the IRP is NP-hard, since the capacitated Vehicle Routing Problem can be represented as a special case of the model. This result largely explains why solving realistic IRP instances remains computationally challenging even today.

Over the years, researchers have proposed a variety of exact optimization methods to address the problem. Archetti et al. (2007), for example, developed a branch-and-cut algorithm for a vendor-managed inventory version of the IRP. Later, Desaulniers et al. (2015) introduced a branch-cut-and-price framework capable of solving instances involving up to 50 customers and 5 vehicles. These studies represent important progress in the literature, but the computational burden of exact methods still increases rapidly as the size of the problem grows. The instances examined in this project involve 12 customers, a planning horizon of 720 hours, and between 60 and 162 driver availability windows, creating a level of temporal complexity that is rarely addressed in exact IRP studies. Even relatively small cases can already require substantial computational effort. For instance, Archetti et al. (2019)

reported average solution times of approximately 1,500 seconds for a 10-node instance solved using a GAMS-based mixed-integer programming model. Considering the longer horizon and operational constraints included in our formulation, depending entirely on exact optimization would not be realistic for this study.

A recent survey by Ekşi et al. (2026) provides a broad overview of modern IRP research and categorizes existing models according to their structure, application area, modeling assumptions, and objective functions. Based on this classification, the LO-IRP developed in this project can be viewed as a multi-vehicle, single-product, deterministic-demand, multi-period IRP with a heterogeneous fleet and a fractional objective function. Variants that combine all these characteristics are relatively uncommon in the literature, especially when long planning horizons and operational driver constraints are included simultaneously. For this reason, the problem is better suited to a customized heuristic framework rather than a purely exact optimization approach.

2.2 Liquid Oxygen and Industrial Gas IRP

The paper from Su et al. (2020), addresses an IRP that is structurally identical to the proposed model in several key respects. The objective function is the same fractional LR formula (operating cost / quantity delivered). The constraints include driver time windows (C18 in the model), safety stock as a hard lower bound on inventory (C3' in the model), lower bounds on delivery quantity, and maximum driving times (C19 in the model). The fleet is heterogeneous with trailer-customer compatibility restrictions (C8 in the model).

Their solution method is a **metaheuristic**: a Metropolis-criterion local search that iteratively improves a feasible solution by combining MIP-based quantity decisions with route construction. While this approach produces high-quality solutions, it requires a commercial MIP solver at each iteration, making it unsuitable for a self-contained educational implementation. We adapt their problem formulation directly but replace the MIP quantity sub-problem with the **coverage-based target quantity formula** described in Section 6 of the algorithm document.

Charitopoulos and Papageorgiou address a Production and Inventory Routing Problem (PIRP) for a real liquid oxygen supply chain in the UK (2022). Their two-level hybrid approach (MILP for production and metaheuristic for distribution) confirms two design choices in our model. First, **stockout prevention for medical oxygen customers must be a hard constraint** (as in C3 in the model), not a penalty term in the objective. Second, safety stock (as in C3' in the model) is the primary sensitivity lever: their numerical experiments show that increasing safety buffers raises LR by 5–15% depending on customer proximity.

He, Artigues, Briand, Jozefowicz, and Ngueveu (2020) address the ROADEF/EURO Challenge 2016 IRP. Their approach seeds a fixed customer-visit sequence derived from a greedy construction, then reoptimizes arrival times and delivered quantities via a mixed-integer fractional program. This two-level decomposition — fixing routing structure, then optimizing quantities and timing — is conceptually related to our algorithm's separation of the route construction loop from the urgency-first quantity assignment. The key difference is that their fixed-sequence sub-problem requires a MILP solver, whereas our quantity assignment is a direct calculation. Their result that arrival-time precision matters for feasibility directly motivates to assign delivery hours exactly and use the arrival-time propagation constraints C16 in the mathematical model.

2.3 IRP with Logistics Ratio Objective

Archetti et al. (2017) introduced the IRP with Logistic Ratio (IRP-LR) as a distinct problem variant. They prove that minimising LR yields fundamentally different solutions from minimising total cost: a cost-minimizing plan may make small, frequent deliveries that inflate LR, while an LR-minimizing plan bundles deliveries to

maximise quantity per unit of cost. This theoretical finding directly justifies our scoring term $q / (\text{extra_cost} + 1)$, which rewards candidates that deliver large quantities for small marginal distance cost.

2.4 Rolling Horizon Methods for Multi-Period IRP

The rolling-horizon approach decomposes a multi-period problem into a sequence of shorter sub-problems, solves them in chronological order, commits the decisions for the current period, and advances. This is the main structure of our algorithm.

Bertazzi et al. (2014) analyze rolling-horizon heuristics for dynamic and stochastic IRP, comparing different look-ahead window lengths. Their central finding is that a moderate look-ahead window of 72–96 hours typically outperforms both shorter and longer windows so that shorter windows produce reactive, costly routes; longer windows over-fill tanks and waste trailer capacity on non-urgent customers. This directly motivates our choice of default coverage hours as 96.

Campbell and Savelsbergh (2004) address the IRP directly and develop efficient insertion heuristics for vendor-managed inventory settings. Their key insight — that computing an optimal insertion position for each candidate customer separately, rather than jointly, reduces computation time from $O(n^4)$ to $O(n^3)$ without sacrificing solution quality — is directly applicable to best insertion position method in the algorithm, which independently tests each insertion position for each candidate customer.

Lin (1965) introduced 2-opt for the Travelling Salesman Problem. Reversing a sub-segment of the route if total distance decreases is the used method in the algorithm. Its extension to VRP is standard and its simplicity is well-suited to the per-window time budget of our rolling-horizon approach.

2.5 Vehicle Routing with Heterogeneous Fleet, Time Windows and Operational Constraints

2.5.1. NP-Hardness of the Routing Sub-Problem

The routing decisions in the LO-IRP form a Heterogeneous Fleet VRP with Time Windows (HFVRPTW), which is NP-hard as a generalization of the CVRP (Barrero et al., 2021). Even without inventory dynamics or a fractional objective, the HFVRPTW cannot be solved optimally in polynomial time. Penna et al. (2013) show that an Iterated Local Search (ILS) with Variable Neighborhood Descent consistently outperforms exact methods on HFVRP instances with more than 50 customers. For our problem, the HFVRP is embedded inside a 720-hour IRP, making the combined problem several orders of magnitude harder than the standalone HFVRP.

2.5.2. Multi-Trip VRP with Driver Work Hours

Seixas and Antunes (2013) study a multi-trip VRP with time windows, accessibility restrictions, and heterogeneous fleet where each vehicle performs multiple routes per day and the driver's total work hours are limited. Their formulation directly corresponds to our constraints C18 (window start/end), C19 (maximum driving duration H_r), and C20 (inter-shift rest ϑ_r). Their column-generation exact approach is applicable to instances with up to 25 customers but requires hours of computation. We adopt their window-by-window processing structure while replacing column generation with the urgency-based insertion loop.

2.5.3. Best-Position Insertion and 2-opt

Solomon (1987) introduced the cheapest-insertion heuristic for the VRPTW: at each step, the unvisited customer whose cheapest insertion position adds the least extra distance is added to the current route. The combination of insertion construction followed by 2-opt improvement is standard in the VRP literature and is used by Archetti et al. (2012) as the routing component of their IRP metaheuristic.

2.5.4. Safety Stock, Service Level, and Constraint C3'

In the standard IRP formulation, the only inventory lower bound is zero (constraint C3). The strengthened constraint C3' — adding a mandatory safety buffer $s_i \geq 0$ for each customer — is the primary mechanism for modelling service-level requirements for medical oxygen customers.

Su et al. (2020) and Charitopoulos et al. (2022) both enforce safety stock as a hard lower bound, arguing that for medical-grade oxygen customers, any inventory below the safety threshold is operationally unacceptable regardless of cost. Our heuristic implements C3' as a soft target: the delivery quantity formula uses **needed = coverage_demand + s_i – current_inventory** so that safety stock is always included in the coverage target.

3 Modeling

The company must plan liquid oxygen deliveries in a Vendor Managed Inventory system. Each route starts at a base, goes to a source to load liquid oxygen, visits compatible customers, and returns to the base. The model decides which driver-window routes are used, which customers are visited, how much is delivered, and when each delivery is done.

3.1 Sets and indices

Symbol	Meaning	Example
B	Set of bases where drivers and trailers start and finish.	$B = \{0\}$
S	Set of sources where trailers load liquid oxygen.	$S = \{1\}$
C	Set of customers with tanks and hourly demand.	$C = \{2, 3, \dots, 13\}$
N	Set of all nodes. $N = B \cup S \cup C$.	$N = \{0, 1, \dots, 13\}$
R	Set of drivers.	$R = \{1, 2\}$
K	Set of trailers.	$K = \{1, 2\}$
W_r	Set of available work windows for driver r, sorted by start time.	$W_1 = \{1, \dots, 30\}$
T	Set of hourly time periods in the planning horizon.	$T = \{1, \dots, 720\}$ (30 days)
A_k	Set of customers accessible by trailer k.	$A_1 = \{2, 3, 5\}$
$i, j, n, s(\text{source})$	Node indices, usually in N.	$i = 1$ may be a source
r	Driver index.	$r = 2$
k	Trailer index.	$k = 1$
w	Work-window index for a driver.	$w = 5$
t	Hourly time-period index.	$t = 24$ means hour 24

3.2 Parameters

Symbol	Meaning	Example
d_{ij}	Distance from node i to node j.	$d_{1,6} = 35 \text{ km.}$
ω_{ij}	Travel time from node i to node j.	$\omega_{6,8} = 40 \text{ min.}$
h_i	Setup/service time at node i. For sources this is loading/setup; for customers this is unloading/setup.	$h_5 = 25 \text{ min.}$
U_i	Maximum tank capacity of customer i.	$U_5 = 20000.$
I_i^0	Initial inventory of customer i.	$I_5^0 = 7000.$
D_{it}	Forecast demand of customer i during hour t.	$D_{5,10} = 900.$
Q_k	Capacity of trailer k.	$Q_1 = 23000.$
c_k^D	Distance cost of trailer k.	$c_1^D = 2 \text{ TL/km.}$
ε_{ik}	Trailer-customer compatibility. Equals 1 if trailer k may visit customer i; 0 otherwise.	<i>If trailer 2 cannot access customer 7, $\varepsilon_{7,2} = 0.$</i>
$k(r)$	Trailer assigned to driver r.	<i>If driver 1 uses trailer 2, $k(1) = 2.$</i>
$b(r)$	Base assigned to driver r.	<i>If driver 1 starts at base 0, $b(1) = 0.$</i>
c_r^T	Time cost of driver r.	$c_1^T = 1.5 \text{ TL/min.}$
H_r	Maximum pure driving time allowed for driver r.	$H_1 = 600 \text{ min.}$
ϑ_r	Minimum rest time after a used shift for driver r.	$\vartheta_1 = 600 \text{ min.}$
a_{rw}	Start time of work window w for driver r, in minutes from horizon start.	$a_{1,2} = 1440.$
b_{rw}	End time of work window w for driver r, in minutes.	$b_{1,2} = 2340.$
ϖ_t	Start minute of hour t.	$\varpi_{10} = 540$ <i>if hour 10 starts at min 540.</i>
$\bar{\varpi}_t$	End minute of hour t. Usually $\bar{\varpi}_t = \varpi_t + 60.$	$\bar{\varpi}_{10} = 600.$
M	Large constant used in conditional constraints.	<i>A safe value is larger than the planning horizon in minutes.</i>
s_i	Safety stock lower bound at customer i. Used in (C3').	<i>Tunable; $s_i = 0$ recovers base model.</i>

3.3 Decision Variables

Variable	Type	Meaning
p_{rw}	$\{0,1\}$	Equals 1 if driver r operates a route in window w . — $p(2,5) = 1 \rightarrow$ driver 2 operates a route in window 5
x_{ijrw}	$\{0,1\}$	Equals 1 if driver r travels directly from node i to node j in window w . — $x(1,7,2,5) = 1 \rightarrow$ driver 2 travels from node 1 to node 7 in window 5
y_{irw}	$\{0,1\}$	Equals 1 if customer i is visited by driver r in window w . — $y(7,2,5) = 1 \rightarrow$ customer 7 is visited by driver 2 in window 5
z_{srw}	$\{0,1\}$	Equals 1 if source s is selected by driver r in window w . — $z(1,2,5) = 1 \rightarrow$ driver 2 uses source 1 in window 5
q_{irwt}	$\{0,1\}$	Equals 1 if customer i 's delivery by driver r in window w is received during hour t . — $q(7,2,5,31) = 1 \rightarrow$ driver 2 delivers to customer 7 in window 5 and it arrives in hour 31
q_{irwt}	≥ 0	Quantity delivered to customer i by driver r in window w during hour t . — $q(7,2,5,31) = 5000 \rightarrow$ driver 2 delivers 5000 units to customer 7 in window 5 during hour 31
I_{it}	≥ 0	Inventory of customer i at the end of hour t . — $I(7,31) = 14000 \rightarrow$ customer 7 has 14000 units at the end of hour 31
A_{nrw}	≥ 0	Arrival time at non-base node n (source or customer) when visited by driver r in window w . — $A(7,2,5) = 1830 \rightarrow$ driver 2 arrives at node 7 at minute 1830 in window 5
S_{rw}	≥ 0	Actual start time of the trip operated by driver r in window w . — $S(1,2) = 1440 \rightarrow$ driver 1 starts the trip in window 2 at minute 1440
E_{rw}	≥ 0	Actual end time of the trip operated by driver r in window w . — $E(1,2) = 1780 \rightarrow$ driver 1 ends the trip in window 2 at minute 1780
u_{irw}	≥ 0	Position of customer i in the sequence of driver r in window w . Used for subtour elimination. — $u(4,2,5) = 1 \rightarrow$ customer 4 is the first stop for driver 2 in window 5

ζ_{rw}	≥ 0	Pure driving time of the trip of driver r in window w . — $\zeta(2,5) = 310 \rightarrow$ driver 2 drives 310 minutes in window 5
φ_{rw}	≥ 0	Total trip duration for driver r in window w , including driving, source loading, and customer service times. — $\varphi(2,5) = 420 \rightarrow$ total trip duration for driver 2 in window 5 is 420 minutes
V_{irw}	≥ 0	Trailer load remaining after driver r services node i in window w . Tracks fill level node-by-node. Used in (C7b). — $V(3,2,5) = 18000 \rightarrow$ after serving node 3, driver 2 still carries 18000 units in window 5

3.4 Objective function

$$\min LR = (\sum_{r,w,i,j} c_{k(r)}^D d_{ij} x_{ijrw} + \sum_{r,w} c_r^T \varphi_{rw}) / \sum_{r,w,i,t} q_{irwt} \quad (\text{OBJ})$$

The numerator is total operating cost: distance cost plus driver time cost. The denominator is total quantity delivered. The model minimizes cost per unit delivered.

3.5 Constraints

3.5.1 Inventory Balance and Bounds

$$(C1) \quad I_{it} = I_{i,t-1} + \sum_{(r,w)} q_{irwt} - D_{it} \quad \forall i \in C, t \in T$$

Inventory at the end of hour t equals previous inventory plus deliveries received during hour t minus demand during hour t .

$$(C2) \quad I_{i0} = I_i^0 \quad \forall i \in C$$

Initial inventory is fixed by the data.

$$(C3) \quad 0 \leq I_{it} \leq U_i \quad \forall i \in C, t \in T$$

Inventory must never fall below zero (stockout) or exceed tank capacity (overflow).

$$(C3') \quad s_i \leq I_{it} \leq U_i \quad \forall i \in C, t \in T$$

Strengthened lower bound: inventory must remain above safety stock s_i . Safety stock will be disregarded for computational purposes.

3.5.2 Delivery Timing

$$(C4) \quad \sum_t q_{irwt} = y_{irw} \quad \forall i \in C, r \in R, w \in W_r$$

If customer i is visited by driver r in window w , exactly one delivery hour is assigned.

$$(C5) \quad q_{irwt} \leq U_i \cdot \varrho_{irwt} \quad \forall i, r, w, t$$

Delivery can only occur during the assigned hour.

$$(C6a) \quad A_{irw} \geq \bar{\omega}_t - M(1 - \varrho_{irwt}) \quad \forall i, r, w, t$$

Arrival cannot occur before the start of the assigned hour.

$$(C6b) \quad A_{irw} \leq \bar{\omega}_t + M(1 - \varrho_{irwt}) \quad \forall i, r, w, t$$

Arrival cannot occur after the end of the assigned hour.

3.5.3 Trailer Capacity and Compatibility

$$(C7) \quad \sum_{(i,t)} q_{irwt} \leq Q_{k(r)} \cdot p_{rw} \quad \forall r, w$$

Total delivered quantity on a route cannot exceed trailer capacity.

$$(C7b-i) \quad V_{srw} = Q_{k(r)} \quad \forall s \in S, r \in R, w \in W_r$$

After loading at the source, the trailer starts completely full.

$$(C7b-ii) \quad V_{jrw} \geq V_{irw} - \sum_t q_{irwt} - M(1 - x_{ijrw})$$

Remaining load after stop j equals remaining load after stop i minus quantity delivered at i .

$$(C7b-iii) \quad V_{irw} \geq 0 \quad \forall i \in C, r, w$$

Remaining trailer load must always be non-negative.

$$(C8) \quad y_{irw} \leq \varepsilon_{i,k(r)} \quad \forall i \in C, r, w$$

A customer may only be visited if the trailer assigned is approved for that customer site.

$$(C9) \quad \sum_i y_{irw} \geq p_{rw} \quad \forall r, w$$

A used route must serve at least one customer.

3.5.4 Route Structure

$$(C10a) \quad \sum_s x_{b(r)srw} = p_{rw} \quad \forall r, w$$

A used route must depart from the driver's assigned base.

$$(C10b) \quad \sum_s z_{srw} = p_{rw} \quad \forall r, w$$

A used route selects exactly one source.

$$(C10c) \ x_{b(r)srw} = z_{srw} \quad \forall s \in S, r, w$$

The arc leaving the base must go to the selected source.

$$(C11) \ \sum_{(j \in C)} x_{sjrw} = z_{srw} \quad \forall s, r, w$$

The source must be followed immediately by exactly one customer.

$$(C12a) \ \sum_j x_{ijrw} = y_{irw} \quad \forall i \in C, r, w$$

Each visited customer has exactly one outgoing arc.

$$(C12b) \ \sum_j x_{jirw} = y_{irw} \quad \forall i \in C, r, w$$

Each visited customer has exactly one incoming arc.

$$(C13) \ \sum_{(i \in C)} x_{i,b(r),rw} = p_{rw} \quad \forall r, w$$

Every used route must return to the assigned base.

$$(C14a) \ x_{iirw} = 0 \quad \forall i, r, w$$

$$(C14b) \ x_{b(r),irw} = 0 \quad \forall i \in C, r, w$$

$$(C14c) \ x_{isrw} = 0 \quad \forall i \in C, s, r, w$$

$$(C14d) \ x_{s,b(r),rw} = 0 \quad \forall s, r, w$$

$$(C14e) \ x_{s,s',rw} = 0 \quad \forall s, s', r, w$$

$$(C14f) \ x_{birw} = x_{ibrw} = 0 \quad \forall b \in B \setminus \{b(r)\}, i, r, w$$

Forbidden arc constraints

Forbidden arcs prevent invalid movements such as self-loops, customer→source, or wrong-base usage.

$$(C15) \ u_{irw} - u_{jrw} + |C| x_{ijrw} \leq |C| - 1 \quad \forall i \neq j \in C, r, w$$

Subtour elimination constraint.

3.5.5 Timing and Driver Rules

$$(C16a) \ A_{srw} \geq S_{rw} + \omega_{b(r),s} - M(1 - z_{srw}) \quad \forall s, r, w$$

Arrival at the source must occur after route start plus travel time from the base.

$$(C16b) \ A_{jrw} \geq A_{srw} + h_s + \omega_{sj} - M(1 - x_{sjrw}) \quad \forall s, j \in C, r, w$$

Arrival at the first customer must occur after source arrival, loading time, and travel time.

$$(C16c) \ A_{jrw} \geq A_{irw} + h_i + \omega_{ij} - M(1 - x_{ijrw}) \quad \forall i \neq j \in C, r, w$$

Arrival at the next customer must account for service time and travel.

$$(C16d) \ E_{rw} \geq A_{irw} + h_i + \omega_{i,b(r)} - M(1 - x_{i,b(r),rw}) \quad \forall i \in C, r, w$$

Route end time must occur after the last customer service and return travel.

$$(C17a) \ \zeta_{rw} = \sum_{(i,j)} \omega_{ij} x_{ijrw} \quad \forall r, w$$

Pure driving time equals total travel time across all used arcs.

$$(C17b) \ \varphi_{rw} = \sum_{(i,j)} \omega_{ij} x_{ijrw} + \sum_s h_s z_{srw} + \sum_i h_i y_{irw} \quad \forall r, w$$

Total route duration equals driving time, loading time, and service time.

$$(C18a) \ a_{rw} p_{rw} \leq S_{rw} \leq b_{rw} p_{rw} \quad \forall r, w$$

Route start time must lie inside the driver's working window.

$$(C18b) \ E_{rw} = S_{rw} + \varphi_{rw} \quad \forall r, w$$

Route end time equals start time plus total route duration.

$$(C18c) \ E_{rw} \leq b_{rw} p_{rw} \quad \forall r, w$$

Route must finish before the work window closes.

$$(C19) \ \zeta_{rw} \leq H_r p_{rw} \quad \forall r, w$$

Driving time cannot exceed the driver's maximum allowed duration.

$$(C20) \ S_{rw'} \geq E_{rw} + \vartheta_r - M(2 - p_{rw} - p_{rw'}) \quad \forall r, w < w'$$

A later route can only start after the previous route ends and required rest is completed.

$$(C21) \ \sum_{(r,w,i,t)} q_{irwt} > 0$$

Total delivered quantity must be strictly positive.

4 Algorithm

4.1. Overview

The algorithm solves an Inventory Routing Problem (IRP) for the distribution of liquid oxygen in a Vendor Managed Inventory (VMI) setting. The goal is to decide which customers to visit, when to visit them, and how much product to deliver, while minimizing the Logistics Ratio (LR) — total operating cost divided by total quantity delivered.

The heuristic processes driver time windows one at a time in chronological order. For each window, it constructs a route by inserting customers based on urgency, improves the route with a local search, assigns delivery quantities, and commits the deliveries. After all windows are processed, a rescue pass handles any remaining stockout risks.

4.2. Time Convention

The algorithm operates on two timescales simultaneously. Routing decisions (travel times, driver windows, arrival times, setup times) are tracked in minutes for precision. Inventory decisions (demand consumption, delivery recording, stockout checks) are tracked in discrete hourly buckets.

The bridge between the two is a simple conversion: when a truck arrives at minute m , the delivery is assigned to hour $h = \lfloor m \div 60 \rfloor$ (integer division). For example, an arrival at minute 87 maps to hour 1 (0-indexed), and an arrival at minute 121 maps to hour 2.

Within each hourly bucket, the inventory balance follows this rule:

$$\text{End-of-hour inventory}(h) = \text{End-of-hour inventory}(h-1) + \text{Delivery}(h) - \text{Demand}(h)$$

A delivery in hour h is added before the demand of hour h is subtracted. This means that if a truck arrives in hour 2, only hour 1 demand has already been consumed; the hour 2 demand is subtracted after the delivery is credited.

4.3. Route Structure

Every route follows the same skeleton: Base $\rightarrow$ Source $\rightarrow$ Customer(s) $\rightarrow$ Base. The truck departs from the base, loads product at the source, visits one or more customers to make deliveries, and returns to the base. Each driver is assigned a specific trailer, and only customers whose sites are compatible with that trailer can be visited.

4.4. Chronological Window Processing

Driver windows are sorted by their start time. The algorithm processes each window in this order. When a driver's route is finalized and committed, the delivered quantities are recorded globally, so subsequent windows see the updated inventory picture. This rolling-horizon approach ensures that later drivers do not duplicate deliveries already made by earlier drivers.

Before using a window, the algorithm checks whether the driver has completed the mandatory minimum rest period since the end of the driver's previous shift. If not, the window is skipped entirely.

4.5. Proactive pre-check

Before the greedy insertion loop begins, the algorithm performs a forward-looking safety check. For each customer compatible with the current trailer, it asks: "Will this customer's inventory fall below its safety stock before the next feasible window (using the same trailer type) can reach it?"

To answer this, the algorithm finds the next future window that uses the same trailer and computes the earliest hour that window could arrive at the customer. It then projects the customer's inventory forward to that hour. If the projected inventory is below the safety stock level, the customer is force-inserted into the current route — it cannot afford to wait.

If no future compatible window exists at all, the customer is also flagged as must visit if it will ever drop below safety stock within the planning horizon.

4.6. Greedy Urgency-Based Insertion

After the proactive customers are inserted, the algorithm enters its main construction loop. In each iteration, it evaluates every unvisited compatible customer and selects the one with the highest score. The scoring formula is:

$$Score = W_1 \times Urgency + Efficiency + Tiebreaker$$

where $W_1 = 1,000,000$ is a large weight ensuring urgency always dominates, $Efficiency = Quantity / (Extra Distance Cost + 1)$, and $Tiebreaker = 10,000 / (Hours Until Stockout + 1)$.

The urgency level of each customer is evaluated at its expected arrival hour to capture the actual inventory state at the time of delivery. Urgency is an integer from 0 to 4:

Level	Condition	Meaning
4	Stockout within 24 hours	Critical
3	Below safety stock within 24h	High
2	Below safety stock within 72h	Medium
1	Below safety stock within 168h	Low
0	Safe beyond 168 hours	No urgency

For each candidate, the algorithm tests every valid insertion position in the current route sequence (between the source and the final base). It picks the position that adds the least extra distance while keeping the route feasible (within the driver's time window and maximum driving duration). The customer with the highest overall score is inserted. This process repeats until no more customers can be feasibly inserted or the trailer has insufficient remaining capacity.

4.7. Delivery Quantity Calculation

The delivery quantity for each customer is determined by several factors. The algorithm computes the customer's inventory at the moment of arrival, the upcoming demand over a coverage window, we chose it as 96 hours, and the customer's safety stock requirement. The target delivery is the amount needed to cover the upcoming demand plus the safety stock deficit, subject to three upper bounds: the remaining trailer capacity, the available tank room (tank capacity minus current inventory), and the overflow check.

The overflow check is a forward simulation: the algorithm walks through every future hour from the delivery hour to the end of the horizon, tracking the peak inventory level. If adding the proposed delivery would cause inventory to exceed the tank capacity at any future point due to other already-committed deliveries arriving later, the delivery quantity is reduced accordingly.

4.8. 2-Opt Route Improvement

After all insertions are complete, the algorithm applies a 2-opt local search to the customer portion of the route. It tries reversing every possible sub-segment of the customer sequence. If a reversal reduces the total route distance and the resulting route remains feasible (within the time window and driving limit), the improvement is accepted. This process runs for up to 3 sweeps.

The base (first node), source (second node), and returning base (last node) are fixed; only the customer nodes between them are candidates for reversal.

4.9. Final Quantity Assignment and Commit

After the 2-opt improvement may have changed the visit order, the algorithm reassigns delivery quantities. Customers in the route are sorted by urgency; highest first and the customer with soonest stockout is selected when urgency levels are the same. The trailer's full load is then allocated sequentially in this urgency order, ensuring that the most critical customers are served first regardless of their position in the route. Once finalized, all deliveries are committed to the global delivery matrix, and the driver's next available time is updated (current route ending time plus the mandatory rest period).

4.10. Rescue Pass

After all driver windows have been processed, the algorithm checks whether any customer still has a projected stockout. If so, it enters a rescue phase. For each at-risk customer, it scans all driver windows again to find the earliest feasible one whose trailer is compatible and can reach the customer before the stockout hour.

4.11. Feasibility Constraints

The algorithm enforces the following hard constraints throughout the route construction process: trailer-customer compatibility (only allowed trailer types can visit each site), sequential trailer load (deliveries are subtracted from remaining capacity in visit order), tank capacity with forward overflow prevention, driver time window (the route must end before the window closes), maximum driving duration per shift, and minimum inter-shift rest period.

4.12. Pseudocode

Algorithm 1 — Main Procedure

1. **Sort** all driver windows by start time (earliest first).
2. **for** each driver window w (in chronological order) **do**
 - 2.1. **Check if** the driver has completed the required rest period.
 if not, **skip** this window **and continue** to the next one.
 - 2.2. **Build** a route **for** this window using Algorithm 2.
 - 2.3. **if** a route was built **then**
 Commit all deliveries to the global inventory record.
 Update the driver's next available time (route **end** + rest period).
 end
- end**
3. **repeat** up to n times:
 - 3.1. Identify all customers that still have a projected stockout.
 - 3.2. **if** none exist, stop.
 - 3.3. **for** each at-risk customer **do**
 Scan all driver windows to **find** the earliest one with a compatible trailer **and** whose driver is available.

if found, **build** a dedicated rescue route using Algorithm 3
customer as the mandatory first stop. **Commit** deliveries.
end
3.4. **if** no rescue route was successfully built in this round, stop.

Algorithm 2 — Build Route for a Single Window

1. Start with the main route: Base → Source → Base.

The trailer is fully loaded at the source.

2. **for** each customer compatible with this trailer **do**

2.1. Estimate the earliest hour the next future window (using the same trailer type) could arrive at this customer.

2.2. **Project** the customer's inventory forward to that hour.

2.3. **if** the projected inventory is below the safety stock level, **or if** no future compatible window exists **and** the customer will eventually drop below safety stock:

insert this customer into the route at the position that adds the least extra distance.

End

3. **repeat while** the trailer has remaining capacity:

3.1. **for** each unvisited compatible customer **do**

a) **Find** the cheapest feasible insertion position in the current route.

b) Determine the arrival hour at that position.

c) **Evaluate** the customer's urgency at that arrival hour.

d) Estimate how much product this customer needs.

e) **Compute** an insertion score

f) **Skip** customers whose deliverable quantity is too small,
unless they are critical.

end

3.2. **Select** the customer with the highest score.

3.3. **Insert** that customer into the route at the best position.

3.4. Subtract the planned delivery from the trailer's remaining capacity.

3.5. **if** no customer can be feasibly inserted, stop.

4. **Apply** 2-opt local search to the customer portion of the route:

Try reversing every sub-segment of the customer visit order.

Accept any reversal that shortens the total distance without violating the time window **or** maximum driving duration.

Repeat for up to 3 sweeps.

5. Reassign delivery quantities after the route order is finalized:

- 5.1. Reorder the customers by urgency (highest first) **for** the purpose of allocating the trailer load.
- 5.2. Walk through customers in this urgency order **and assign** each one as much product as it needs, limited by:
 - the remaining trailer capacity at that point,
 - the available room in the customer's tank,
 - the overflow **check** (no future hour can exceed tank capacity).
- 5.3. **Remove** any customer whose delivery is too small to be worthwhile (unless it is critical). Retry the allocation with the cleaned route.

6. **Return** the finalized route with its deliveries.

Algorithm 3 — Rescue Route

1. Start with the route: Base → Source → Rescue Customer → Base. **if** this route is **not** feasible within the time window, **return** failure.
2. Allocate the maximum possible quantity to the rescue customer, limited by trailer capacity, tank room, **and** the overflow check.
3. **if** trailer capacity remains, fill it with other urgent customer using the same greedy insertion loop as Algorithm 2, Step B.
4. **Apply** 2-opt improvement **and** reassign quantities (same as Algorithm 2, Steps C **and** D).
5. **Return** the finalized rescue route.

5 Computational Results

Table: Algorithmic Performance Across Test Instances

Instance	Routes	Quantity (Qty)	Cost	Logistics Ratio (LR)	Stockout (h)	Overflow (h)	Safety Viol. (h)	Runtime (s)
V_1.1	60	1,066,444	45,725	0.042876	0	0	9	0.7
V_1.2	60	1,083,973	45,400	0.041883	0	0	18	0.6
V_1.3	8	162,559	3,219	0.019804	0	0	0	0.3
V_1.4	17	223,012	5,745	0.025761	0	0	36	0.4
V_1.5	16	277,256	5,696	0.020544	0	0	0	0.6
V_1.6	60	724,533	22,474	0.031018	0	0	0	5.1

V_1.7	48	647,303	12,777	0.019739	0	0	134	2.3
V_1.8	18	265,188	4,435	0.016724	0	0	6	0.5
V_1.9	162	1,960,347	51,436	0.026238	0	0	0	22.6
V_1.10	24	250,918	8,334	0.033215	0	0	3	0.7
V_1.11	90	751,575	34,157	0.045447	584	0	1401	6.1
Total	-	-	-	-	-	-	-	39.9

The total runtime across all 11 instances was 39.9 seconds, with an average of approximately 3.6 seconds per instance. For smaller instances such as V_1.3, V_1.4 and V_1.5 the run time was almost instantaneous lasting less than a second. Most notable is Instance V_1.9 with 162 routes and nearly 2 million units so the runtime scaled to 22.6 seconds. This demonstrates that the algorithm can handle high density logistics problems within reasonable timeframes.

The model achieved 0% overflow rate across all tested scenarios, indicating that the routing logic respects storage constraints perfectly. Additionally, 10 out of 11 instances achieved zero stockouts which suggests high reliability in meeting demand under standard operational parameters. The single problem can be the safety stock violation in 7 out of 11 instances, but they are mostly negligible (less than 10 hours).

A single problematic instance is V_1.11 as it has stockout hours and many safety violations, but in this specific instance the problem was with the data, as some customers were unreachable, which caused these inaccuracies. In terms of logistic ratio, results were relatively stable across the runs, ranging between 0.016 and 0.042, and even for the largest instance (V_1.9), the algorithm maintained a highly efficient LR of 0.026. This shows that the algorithm manages to obtain good results for any type of data.

6 Sensitivity Analysis

As we have conducted comprehensive sensitivity analysis, we have included the most important points in this report. All other results can be reached in the python file provided.

Below, the table shows the maximum and minimum LR acquired during the sensitivity analysis. For some instances, the change was insignificant with the change of 1 % or even less. However, there were instances and parameters resulting in significant percent change.

safety_stock:

Instance	min mult	min LR	baseline	max mult	max LR	%-change
Instance_V_1.1	0.50	0.04320	0.04288	1.50	0.04268	-1.22%
Instance_V_1.2	0.50	0.04194	0.04188	1.50	0.04217	0.55%
Instance_V_1.3	0.50	0.02238	0.01980	1.50	0.02438	10.09%
Instance_V_1.4	0.50	0.02428	0.02576	1.50	0.02973	21.16%
Instance_V_1.5	0.50	0.02073	0.02054	1.50	0.02228	7.53%
Instance_V_1.6	0.50	0.03106	0.03102	1.50	0.03120	0.44%
Instance_V_1.7	0.50	0.01891	0.01974	1.50	0.02260	18.68%
Instance_V_1.8	0.50	0.01676	0.01672	1.50	0.01771	5.68%
Instance_V_1.9	0.50	0.02572	0.02624	1.50	0.02662	3.44%
Instance_V_1.10	0.50	0.03240	0.03321	1.50	0.03641	12.07%
Instance_V_1.11	0.50	0.04499	0.04545	1.50	0.04662	3.59%

trailer_capacity:

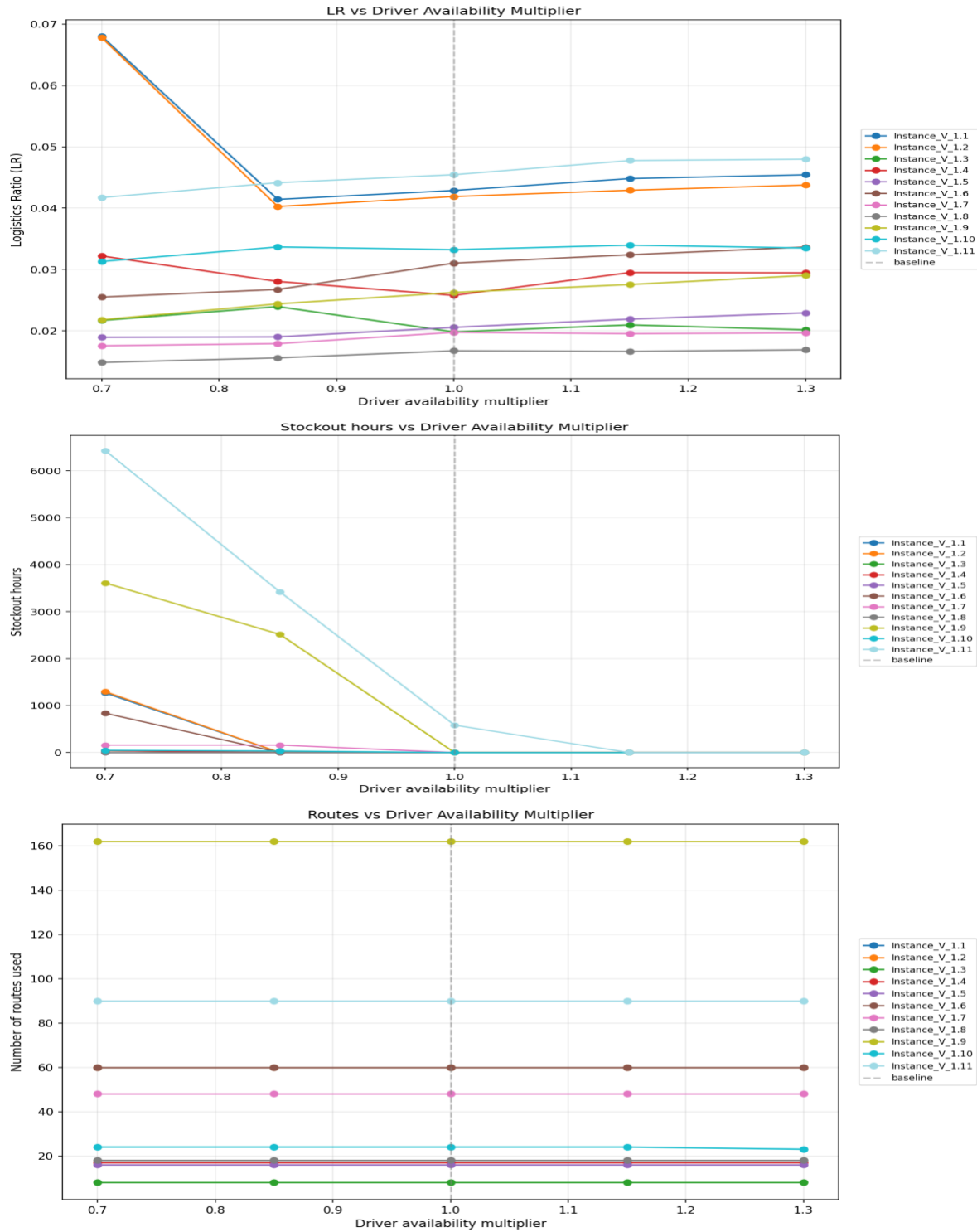
Instance	min mult	min LR	baseline	max mult	max LR	%-change
Instance_V_1.1	0.70	0.04269	0.04288	1.30	0.04123	-3.41%
Instance_V_1.2	0.70	0.04306	0.04188	1.30	0.04034	-6.50%
Instance_V_1.3	0.70	0.02425	0.01980	1.30	0.02044	-19.26%
Instance_V_1.4	0.70	0.04161	0.02576	1.30	0.02330	-71.09%
Instance_V_1.5	0.70	0.02046	0.02054	1.30	0.02184	6.72%
Instance_V_1.6	0.70	0.02982	0.03102	1.30	0.03097	3.72%
Instance_V_1.7	0.70	0.02438	0.01974	1.30	0.02003	-22.06%
Instance_V_1.8	0.70	0.01981	0.01672	1.30	0.01520	-27.58%
Instance_V_1.9	0.70	0.02380	0.02624	1.30	0.02863	18.40%
Instance_V_1.10	0.70	0.03810	0.03321	1.30	0.03345	-13.99%
Instance_V_1.11	0.70	0.04395	0.04545	1.30	0.04671	6.07%

driver_availability:

Instance	min mult	min LR	baseline	max mult	max LR	%-change
Instance_V_1.1	0.70	0.06804	0.04288	1.30	0.04543	-52.74%
Instance_V_1.2	0.70	0.06780	0.04188	1.30	0.04377	-57.39%
Instance_V_1.3	0.70	0.02167	0.01980	1.30	0.02014	-7.70%
Instance_V_1.4	0.70	0.03218	0.02576	1.30	0.02944	-10.67%
Instance_V_1.5	0.70	0.01892	0.02054	1.30	0.02292	19.44%
Instance_V_1.6	0.70	0.02549	0.03102	1.30	0.03365	26.30%
Instance_V_1.7	0.70	0.01755	0.01974	1.30	0.01965	10.63%
Instance_V_1.8	0.70	0.01483	0.01672	1.30	0.01689	12.31%
Instance_V_1.9	0.70	0.02178	0.02624	1.30	0.02902	27.58%
Instance_V_1.10	0.70	0.03129	0.03321	1.30	0.03349	6.62%
Instance_V_1.11	0.70	0.04172	0.04545	1.30	0.04798	13.76%

Total wall-clock time: 866 sec (14.4 min)

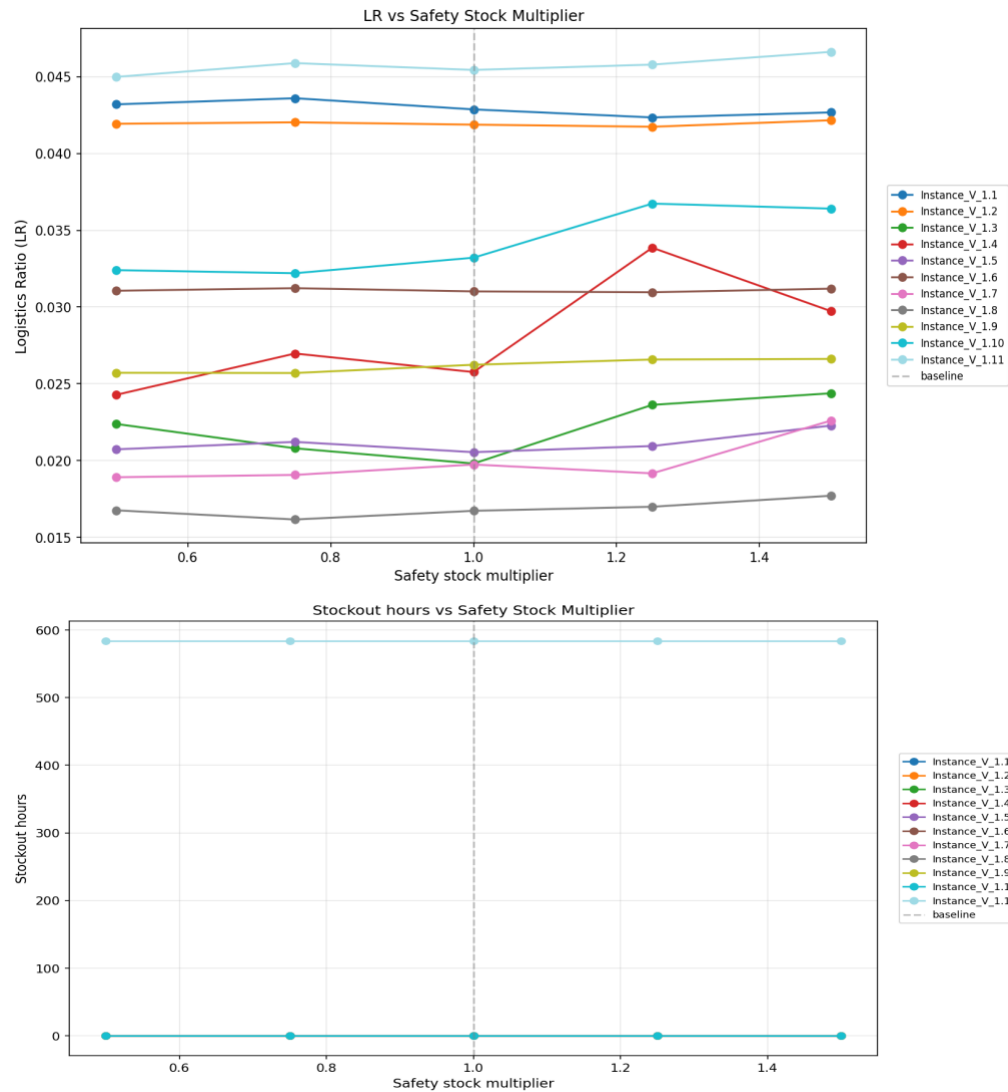
6.1 Driver Availability Sensitivity

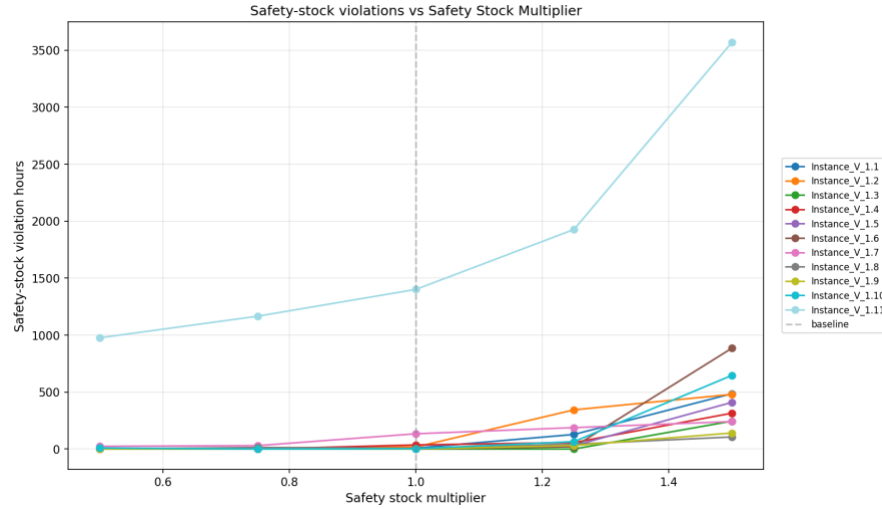


We are multiplying driver availability by factors of 0.7, 0.85, 1, 1.15 and 1.3. From these graphs, we can point out that as driver availability increases, stockout hours approach zero, so they have some inverse relationship. The number of routes doesn't change (maybe for higher multipliers, these parameters would change as they need to be somehow

related to each other), but LR has unpredictable results, sometimes increasing when driver availability increases, and sometimes decreasing when driver availability increases. It is most probably related to the nature of the algorithm as its heuristic based algorithm. It is "noise" of a heuristic trying to balance inventory stability against transportation efficiency resulting in complex trade-offs.

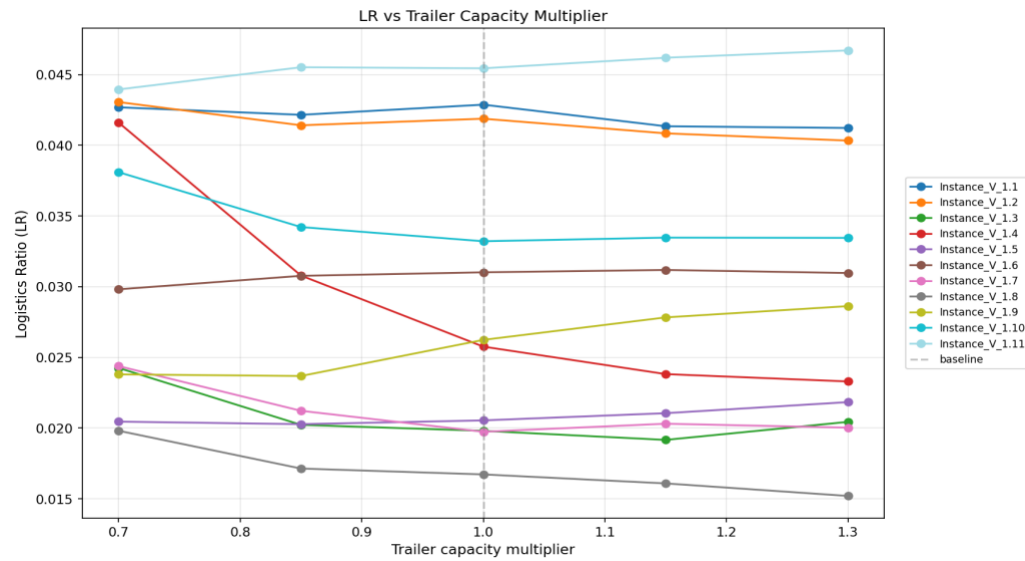
6.2 Safety Stock Sensitivity

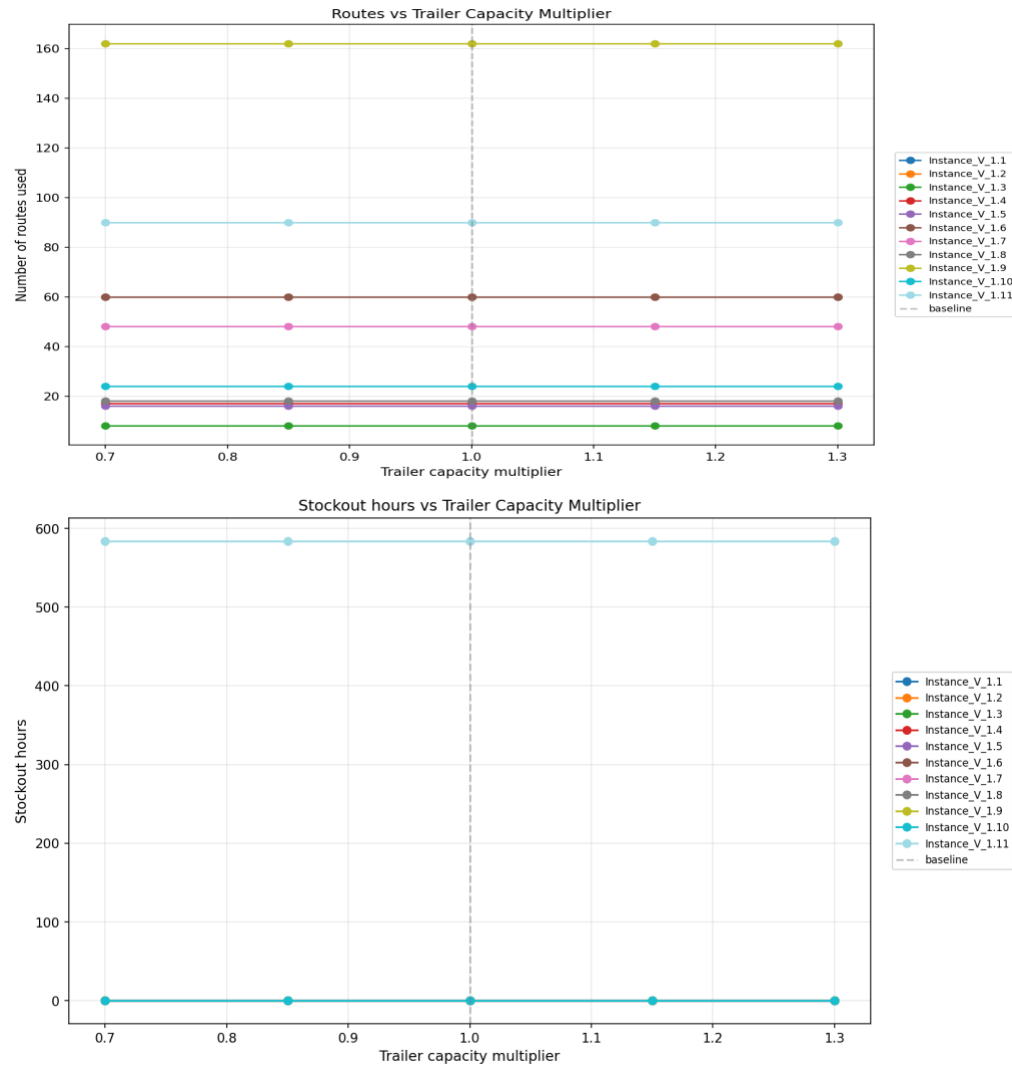




We are multiplying safety stocks by factors of 0.5, 0.75, 1, 1.25 and 1.5. From these graphs, we can point out that as safety stock increases, safety-stock violations increase, so they have some direct relationship. Stockout hours always stay the same as they are not related to safety stock. Logistic ratio has unpredictable results sometimes increasing when safety stock increases and sometimes decreasing when safety stock increases. It is most probably related to the nature of the algorithm as its heuristic based algorithm. It is "noise" of a heuristic trying to balance inventory stability against transportation efficiency resulting in complex trade-offs.

6.3 Trailer Capacity Sensitivity





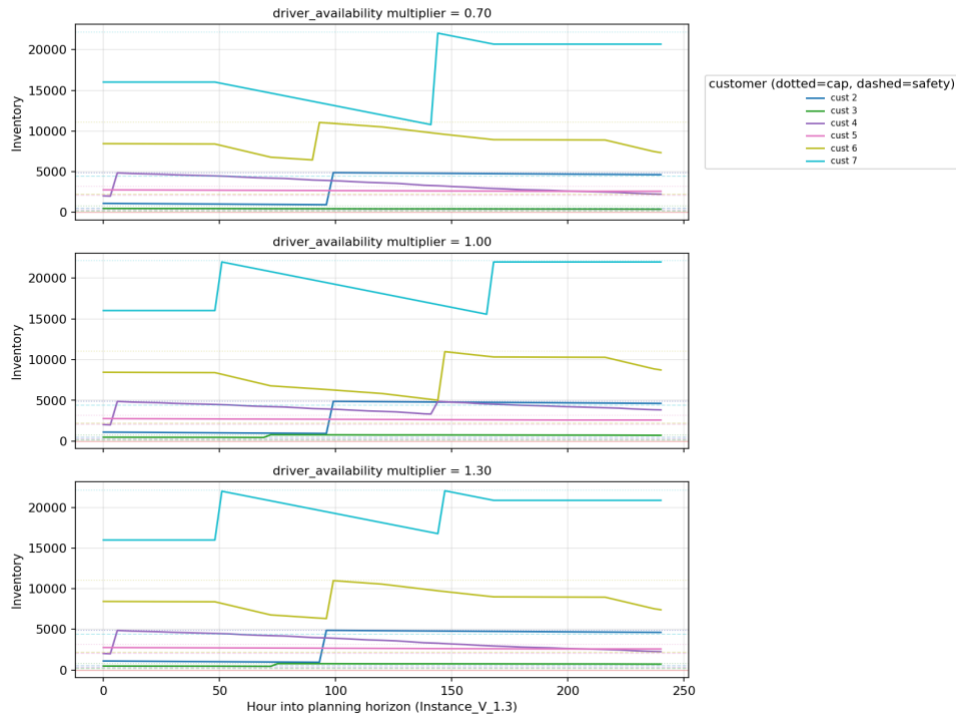
We are multiplying trailer capacities by factors of 0.7, 0.85, 1.15 and 1.3. From these graphs, we can point out that the number of routes and stockout hours stay the same for these values of trailer capacities (maybe for higher multipliers, these parameters would change as they need to be somehow related to each other). Logistic ratio has unpredictable results sometimes increasing when safety stock increases and sometimes decreasing when safety stock increases. It is most probably related to the nature of the algorithm as its heuristic based algorithm. It is "noise" of a heuristic trying to balance inventory stability against transportation efficiency resulting in complex trade-offs.

6.4 Inventory Levels Sensitivity

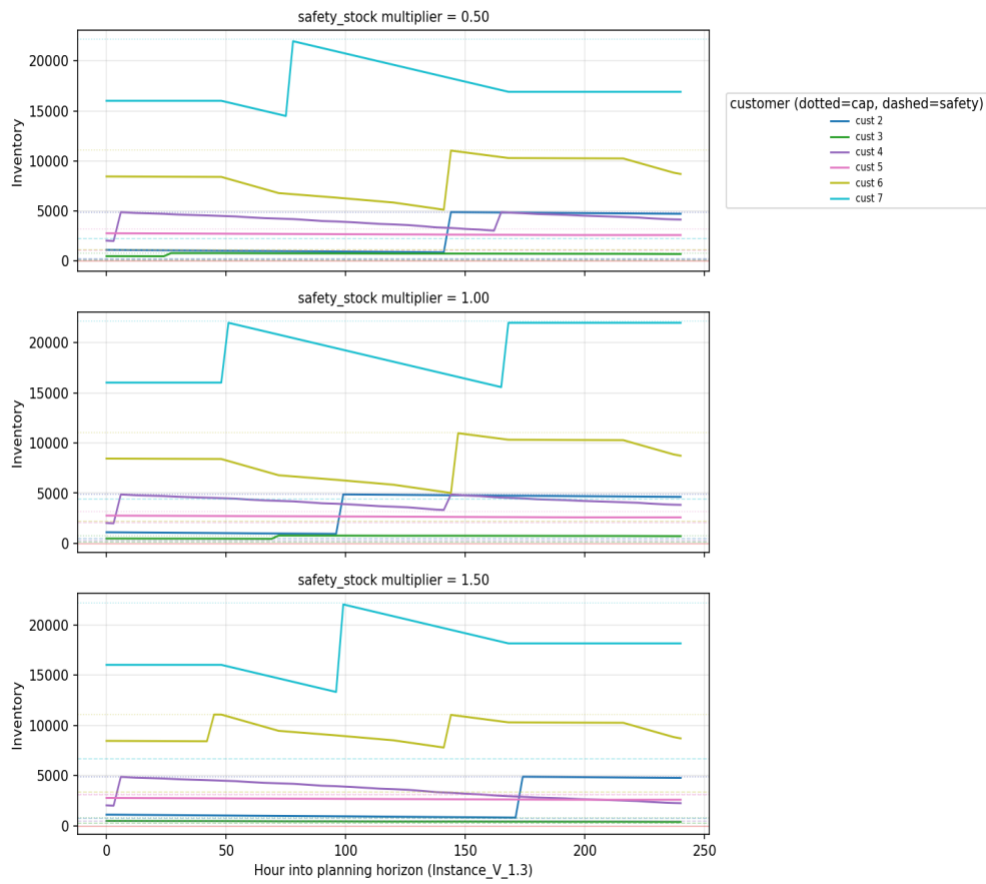
We have included the inventory sensitivity analysis only for 2 instances, other instances can be reached via the code output.

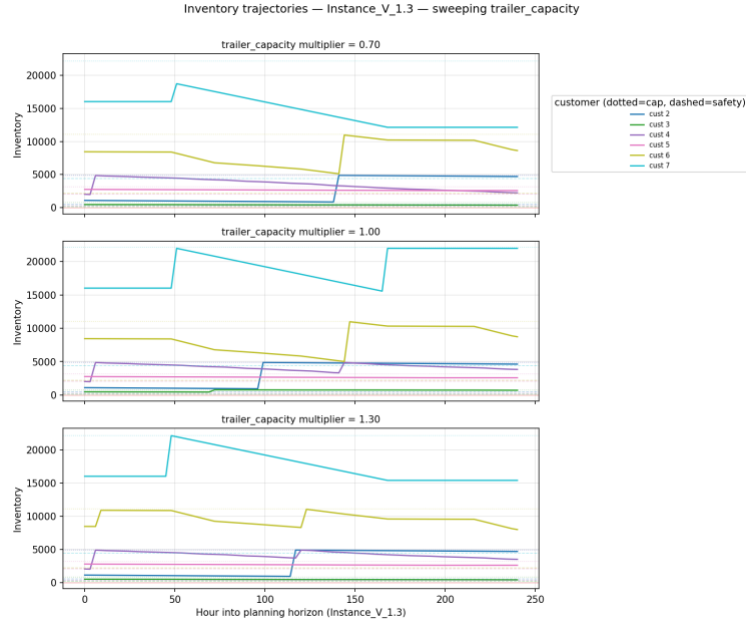
Instance V_1.3:

Inventory trajectories — Instance_V_1.3 — sweeping driver_availability



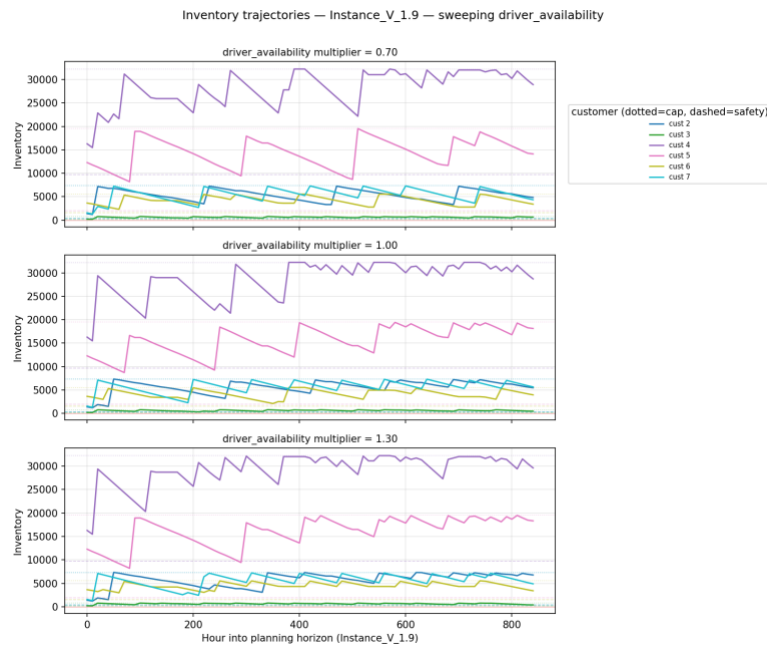
Inventory trajectories — Instance_V_1.3 — sweeping safety_stock



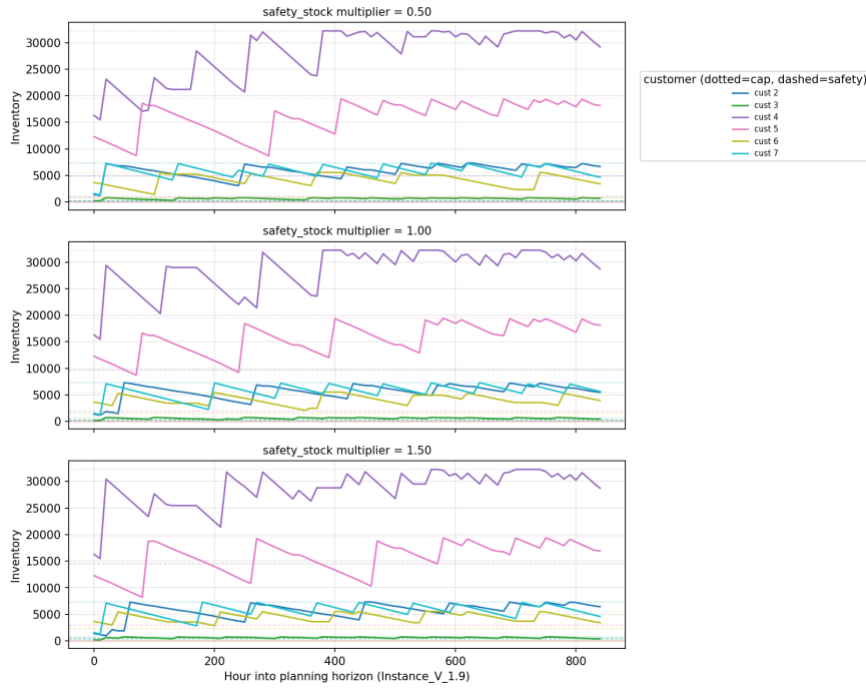


These charts visualize how inventory is being managed for a V_1.3 scenario, illustrating how the system responds to shifts in driver availability, safety stock, and trailer capacity. This instance shows remarkable stability, where even significant reductions in resources do not cause the system to enter a deficit. Instead, the heuristic adapts to the instance by slightly changing the timing and volume of deliveries to keep inventory floating consistently near safety stock levels. This indicates a robust operational configuration where the constraints are easily managed within the system's limits.

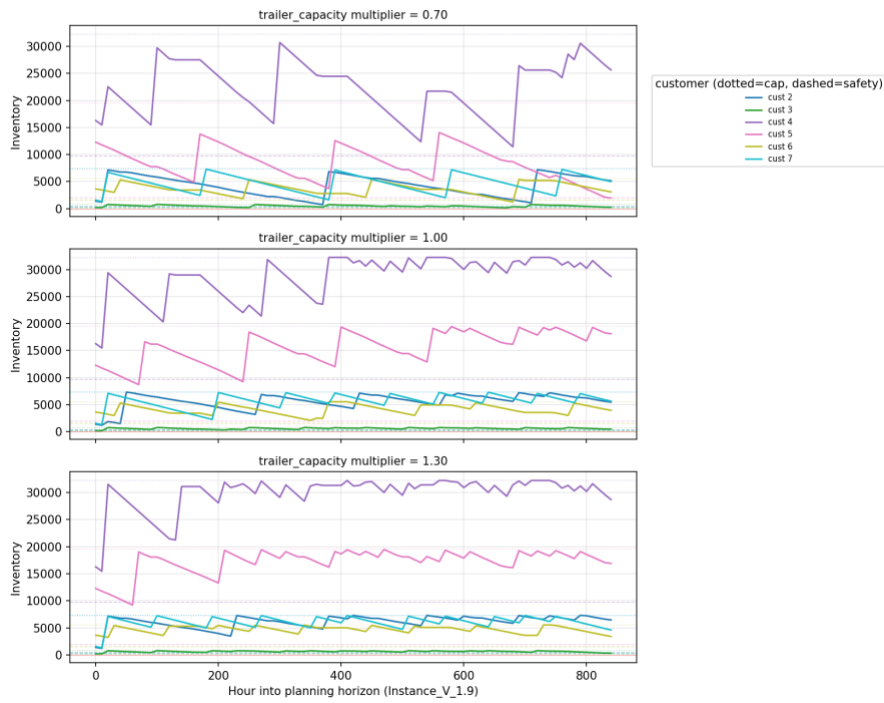
Instance V_1.9:



Inventory trajectories — Instance_V_1.9 — sweeping safety_stock



Inventory trajectories — Instance_V_1.9 — sweeping trailer_capacity



The same comments apply to this instance as there is a remarkable stability, despite the huge size of data for this instance.

7 Conclusion

This study examines the Liquid Oxygen Inventory Routing Problem under a Vendor Managed Inventory (VMI) framework, where maintaining continuous product availability is essential due to the direct impact of oxygen shortages on patient safety in healthcare facilities. The complexity of the problem is caused by the need for an effective distribution plan over a 30-day planning horizon while simultaneously coordinating inventory management, vehicle routing, and workforce scheduling under strict physical, operational, and temporal constraints. As the problem contains the Heterogeneous Fleet Vehicle Routing Problem with Time Windows (HFVRPTW), which is NP-hard, and as exact optimization methods are computationally impractical for a 720-hour planning horizon, relying only on exact approaches is not feasible. To address this issue, a customized rolling-horizon heuristic was developed with the objective of minimizing the Logistics Ratio (LR), a fractional performance measure that balances route efficiency and delivery effectiveness to achieve the lowest possible delivery cost per unit.

The proposed heuristic processes driver availability windows sequentially and employs a proactive forward-looking safety mechanism together with a greedy urgency-based insertion procedure to construct feasible delivery routes. Customer urgency is dynamically assessed according to projected inventory levels, while route quality is improved using a 2-opt local search procedure. In addition, the separation of route construction from the urgency-based quantity allocation mechanism enables the model to satisfy critical medical oxygen demand as a hard constraint, ensuring that stockout prevention remains the highest operational priority.

Overall, the proposed heuristic framework provides a practical and scalable solution for a highly constrained real-world logistics environment. By implementing fleet compatibility requirements, strict driver work regulations, and inventory constraints into a single unified model, the approach offers companies such as Air Liquide an effective strategy for reducing operational costs while preserving the reliability and continuity of their cryogenic supply chain operations.

8 References

André, J., Bourreau, E., & Wolfler Calvo, R. (2020). Introduction to the ROADEF/EURO Challenge 2016

special section on inventory routing. *Transportation Science*, 54(2), 299–301.

<https://doi.org/10.1287/trsc.2020.0975>

Archetti, C., Bertazzi, L., Hertz, A., & Speranza, M. G. (2012). A hybrid heuristic for an inventory routing problem. *INFORMS Journal on Computing*, 24(1), 101–116. <https://doi.org/10.1287/ijoc.1100.0439>

Archetti, C., Bertazzi, L., Laporte, G., & Speranza, M. G. (2007). A branch-and-cut algorithm for a vendor-managed inventory-routing problem. *Transportation Science*, 41(3), 382–391.

<https://doi.org/10.1287/trsc.1060.0188>

- Archetti, C., Coelho, L. C., & Speranza, M. G. (2019). An exact algorithm for the inventory routing problem with logistic ratio. *Transportation Research Part E*, 131, 96–107. <https://doi.org/10.1016/j.tre.2019.09.016>
- Archetti, C., Desaulniers, G., & Speranza, M. G. (2017). Minimizing the logistic ratio in the inventory routing problem. *EURO Journal on Transportation and Logistics*, 6(4), 289–306. <https://doi.org/10.1007/s13676-016-0097-9>
- Barrero, L., et al. (2021). A GRASP/VND heuristic for the heterogeneous fleet vehicle routing problem with time windows. *Lecture Notes in Computer Science*, 12559, 152–165. https://doi.org/10.1007/978-3-030-69625-2_12
- Bertazzi, L., et al. (2014). Heuristics for dynamic and stochastic inventory-routing. *Computers & Operations Research*, 52, 55–67. <https://doi.org/10.1016/j.cor.2014.07.001>
- Campbell, A., Clarke, L., Kleywegt, A., & Savelsbergh, M. (1998). The inventory routing problem. In T. G. Crainic & G. Laporte (Eds.), *Fleet management and logistics* (pp. 95–113). Springer. https://doi.org/10.1007/978-1-4615-5755-5_4
- Campbell, A. M., & Savelsbergh, M. W. P. (2004). Efficient insertion heuristics for vehicle routing and scheduling problems. *Transportation Science*, 38(3), 369–378. <https://doi.org/10.1287/trsc.1030.0046>
- Charitopoulos, V. M., & Papageorgiou, L. G. (2022). Integrated production and inventory routing planning of oxygen supply chains. *Chemical Engineering Research and Design*, 185, 81–97. <https://doi.org/10.1016/j.cherd.2022.07.027>
- Desaulniers, G., Rakke, J. G., & Coelho, L. C. (2015). A branch-price-and-cut algorithm for the inventory-routing problem. *Transportation Science*, 50(3), 1060–1081. <https://doi.org/10.1287/trsc.2015.0621>
- Ekşi, A., Elyasi, M., Minner, S., van Woensel, T., Örsan Özener, O., & Ekici, A. (in press). Inventory-routing problems: Review, challenges, and innovations. *European Journal of Operational Research*. <https://doi.org/10.1016/j.ejor.2026.04.044>
- Govindan, K. (2013). Vendor-managed inventory: a review based on dimensions. *International Journal of Production Research*, 51(13), 3808–3835. <https://doi.org/10.1080/00207543.2012.751511>

He, Y., Artigues, C., Briand, C., Jozefowicz, N., & Ngueveu, S. U. (2020). A matheuristic with fixed-sequence reoptimization for a real-life inventory routing problem. *Transportation Science*, 54(2), 355–374.

<https://doi.org/10.1287/trsc.2019.0954>

Lin, S. (1965). Computer solutions of the traveling salesman problem. *Bell System Technical Journal*, 44(10), 2245–2269. <https://doi.org/10.1002/j.1538-7305.1965.tb04146.x>

Penna, P. H. V., Subramanian, A., & Ochi, L. S. (2013). An iterated local search heuristic for the heterogeneous fleet vehicle routing problem. *Journal of Heuristics*, 19(2), 201–232.

<https://doi.org/10.1007/s10732-011-9186-y>

Seixas, M. P., & Antunes, A. P. (2013). Column generation for a multi-trip VRP with time windows, driver work hours, and heterogeneous fleet. *Mathematical Problems in Engineering*, 2013.

<https://doi.org/10.1155/2013/824961>

Solomon, M. M. (1987). Algorithms for the vehicle routing and scheduling problems with time window constraints. *Operations Research*, 35(2), 254–265. <https://doi.org/10.1287/opre.35.2.254>

Su, Z., Lü, Z., Wang, Z., Qi, Y., & Benlic, U. (2020). A matheuristic algorithm for the inventory routing problem. *Transportation Science*, 54(2), 330–354. <https://doi.org/10.1287/trsc.2019.0930>

Govindan, K. (2013). Vendor-managed inventory: A review based on dimensions. *International Journal of Production Research*, 51(13), 3808–3835. <https://doi.org/10.1080/00207543.2012.751511>